Role-based registrar approval matrix: map department draft, Academic Affairs, and final publish permissions by role

Computer
Registrar state machine spec: transition rules, validation hooks, and audit logging for course schedule publishing
Computer
Show the database schema for this workflow state machine
Show the API endpoints for state transitions and validation
Map out the audit logging and FERPA tracking requirements

Below is a production-style spec for the **registrar approval matrix, workflow schema, APIs, and FERPA-grade audit model** for Semester's course-schedule publishing flow. Registrar workflows in higher education often route department submissions through structured approvals, send special cases to Academic Affairs, and then move approved records into the official registrar publishing path. [1] [2] [3]

RBAC approval matrix

The permission model should separate workflow stage authority from raw data-edit authority, and it should distinguish **view**, **edit**, **approve**, **export**, and **override** rights. RBAC best practices emphasize least privilege, role-scoped permissions, changelogs for access rules, and logging for access and changes. [4] [5]

Workflow roles

Role	Primary scope	Main responsibility
Department scheduler	Department/program	Build sections and submit them into workflow
Department chair	Department/program	Academic and staffing approval
Dean / associate dean	College/school	College-level approval and exception sponsorship
Academic Affairs reviewer	Institution/college	Review policy-sensitive and non-standard requests
Registrar scheduler	Institution/campus	Central schedule review, validation, and routing
Registrar admin	Institution/campus	Final publish, lock, override, emergency correction
Room manager	Room pool / campus	Central room assignment and room conflict resolution
Auditor / compliance reviewer	Scoped read-only	Audit evidence review only

Permission matrix by role

Capability	Dept scheduler	Chair	Dean	Academic Affairs	Registrar scheduler	Registrar admin	Room manager	Auditor
View department drafts	Yes	Yes	Yes	Scoped	Yes	Yes	Scoped	Scoped
Edit draft section data	Yes	No	No	No	Limited	Yes	No	No
Submit for validation	Yes	Yes	Yes	No	Yes	Yes	No	No
Return to department	No	Yes	Yes	Yes	Yes	Yes	Yes, room-only	No
Approve assigned stage	No	Yes	Yes	Yes	Yes	Yes	No	No

Capability	Dept scheduler	Chair	Dean	Academic Affairs	Registrar scheduler	Registrar admin	Room manager	Auditor
Route to Academic Affairs	Limited	Yes	Yes	No	Yes	Yes	No	No
Deny exception	No	No	Limited	Yes	Limited	Yes	No	No
Assign department room	Scoped	No	No	No	Yes	Yes	Yes	No
Assign central room	No	No	No	No	Yes	Yes	Yes	No
Override blocking conflict	No	No	No	Limited	Limited	Yes	Limited	No
Publish official schedule	No	No	No	No	No	Yes	No	No
Lock published record	No	No	No	No	Limited	Yes	No	No
Export scoped records	Limited	Limited	Limited	Limited	Yes	Yes	Limited	Limited
View audit events	No	No	Limited	Limited	Limited	Yes	No	Yes
View FERPA disclosure records	No	No	No	Limited	Limited	Yes	No	Yes

State-sensitive matrix

Workflow state	Dept scheduler	Chair	Dean	Academic Affairs	Registrar scheduler	Registrar admin
`draft`	Edit, save, submit	View	View	View	View	View
`validation_pending`	View	View	View	View	View	View
`validation_failed`	Edit, resubmit, request exception	View/comment	View/comment	View	View/comment	View/comment
`department_ready`	View	Approve/return	View	View	View	View
`chair_review`	View/comment	Approve/return	View	View	View	View
`dean_review`	View	Comment	Approve/return	View	View	View
`registrar_review`	View	View	View	View	Approve/return/route	Override
`room_coordination`	View/comment	View	View	View	Assign/return	Override
`academic_affairs_review`	View	View	View	Approve/deny/return	Comment	Override
`approved_for_publish`	View	View	View	View	View	Publish/return
`published`	Request change	Request change	Request change	View	Request change	Lock/unpublish policy action

Least-privilege access is essential because education systems should allow users to see only what their role requires, and FERPA-oriented software guidance specifically calls for separate permissions for viewing, editing, and exporting data.[6] [7]

Workflow state machine

Structured registrar workflows reduce paper processes, route approvals consistently, and support official records publication. Academic Affairs approval is especially important for non-standard or special class requests in some university workflows.[8] [1]

States

```
draft
validation_pending
validation_failed
department_ready
chair_review
dean_review
registrar_review
room_coordination
academic_affairs_review
approved_for_publish
published
change_requested
locked
denied
archived
```

Transition rules

From state	Action	To state	Rule
`draft`	Submit	`validation_pending`	Required fields present
`validation_pending`	Pass validation	`department_ready`	No blocking conflicts
`validation_pending`	Fail validation	`validation_failed`	Blocking rule or missing data
`validation_failed`	Resubmit	`validation_pending`	Errors corrected
`validation_failed`	Request exception	`academic_affairs_review`	Exception-eligible issue
`department_ready`	Chair approves	`chair_review` or immediate `dean_review`	Configurable workflow path
`chair_review`	Approve	`dean_review`	Chair approval recorded
`chair_review`	Return	`draft`	Comments required
`dean_review`	Approve	`registrar_review`	College approval recorded
`dean_review`	Route exception	`academic_affairs_review`	Policy-sensitive case
`registrar_review`	Approve	`room_coordination` or `approved_for_publish`	Depends on room assignment need
`registrar_review`	Return	`draft`	Registrar notes required
`registrar_review`	Route exception	`academic_affairs_review`	Non-standard or restricted case
`room_coordination`	Assign and clear	`approved_for_publish`	Room and resource checks passed
`room_coordination`	Return	`registrar_review` or `draft`	Conflict unresolved
`academic_affairs_review`	Approve	`registrar_review`	Exception accepted
`academic_affairs_review`	Deny	`denied`	Written reason required
`approved_for_publish`	Publish	`published`	Snapshot + audit event required
`published`	Request change	`change_requested`	Post-publish change flow
`change_requested`	Reopen	`validation_pending` or `registrar_review`	Depends on change severity
`published`	Lock	`locked`	Official lock action
`locked`	Emergency unlock	`registrar_review`	Registrar admin only

From state	Action	To state	Rule
`denied`	Rework	`draft`	New submission cycle
any terminal	Archive	`archived`	Retention policy path

Validation hooks

Validation should run:

- On draft save, for soft warnings.
- On submit, for blocking errors.
- On each approval transition, for stale-data checks.
- Before publish, for full certification.

Required validation categories:

- Required fields complete.
- Approved meeting pattern or documented exception.
- Instructor overlaps.
- Room overlaps.
- Room-capacity mismatch.
- Contact-hour mismatch.
- Cross-list consistency.
- Publishability checks such as required approvals and missing room status.

Registrar scheduling policies frequently require standard meeting patterns to reduce scheduling conflicts and improve room utilization. [9] [10] [11]

API endpoints

The API should be split into data endpoints, workflow-transition endpoints, validation endpoints, and audit endpoints.

Section and workflow endpoints

```
GET    /api/terms/{termId}/sections
POST   /api/terms/{termId}/sections
GET    /api/sections/{sectionId}
PATCH  /api/sections/{sectionId}
POST   /api/sections/{sectionId}/submit
POST   /api/sections/{sectionId}/resubmit
POST   /api/sections/{sectionId}/request-exception
POST   /api/sections/{sectionId}/request-change
POST   /api/sections/{sectionId}/lock
POST   /api/sections/{sectionId}/unlock-emergency
```

Review and approval endpoints

```
GET    /api/workflow/tasks?assignee=me
GET    /api/workflow/instances/{instanceId}
POST   /api/workflow/tasks/{taskId}/approve
POST   /api/workflow/tasks/{taskId}/return
POST   /api/workflow/tasks/{taskId}/deny
POST   /api/workflow/tasks/{taskId}/route-academic-affairs
POST   /api/workflow/tasks/{taskId}/comment
```

Validation and conflict endpoints

```
POST   /api/sections/{sectionId}/validate
GET    /api/sections/{sectionId}/conflicts
GET    /api/conflicts?termId=&orgUnitId=&severity=&status=
POST   /api/conflicts/{conflictId}/waive
POST   /api/conflicts/{conflictId}/resolve
GET    /api/rooms/availability?termId=&day=&start=&end=&capacity=
POST   /api/sections/{sectionId}/assign-room
```

Publish endpoints

```
POST   /api/terms/{termId}/prepublish-check
GET    /api/terms/{termId}/publication-preview
POST   /api/terms/{termId}/publish
POST   /api/terms/{termId}/enter-refine-mode
POST   /api/terms/{termId}/final-lock
```

Audit and FERPA endpoints

```
GET    /api/audit-events?targetType=section&targetId=
GET    /api/audit-events?actorId=&module=registrar
GET    /api/access-reviews?scopeType=&scopeId=
POST   /api/exports/sections
POST   /api/ferpa/disclosure-requests
GET    /api/ferpa/disclosure-requests/{id}
POST   /api/ferpa/disclosure-requests/{id}/decide
```

A practical rule is that every workflow transition endpoint should generate both a transition record and an audit event, while sensitive read/export endpoints should generate access-log events as well. FERPA-oriented software guidance emphasizes logging access, edits, exports, permission changes, and disclosure decisions.[12] [6]

Database schema

The schema should keep **academic records**, **workflow records**, **RBAC records**, and **audit/disclosure records** as distinct domains.

Academic and workflow schema

```
CREATE TABLE terms (
  id uuid PRIMARY KEY,
  code text UNIQUE NOT NULL,
  name text NOT NULL,
  starts_on date NOT NULL,
  ends_on date NOT NULL,
  state text NOT NULL CHECK (
    state IN (
      'draft','validation_pending','validation_failed','department_ready',
      'chair_review','dean_review','registrar_review','room_coordination',
      'academic_affairs_review','approved_for_publish','published',
      'change_requested','locked','denied','archived'
    )
  ),
  publish_version integer NOT NULL DEFAULT 0,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE courses (
  id uuid PRIMARY KEY,
  subject_code text NOT NULL,
```

```
  catalog_number text NOT NULL,
  title text NOT NULL,
  credit_hours numeric(4,2) NOT NULL,
  owning_org_unit_id uuid NOT NULL,
  UNIQUE(subject_code, catalog_number)
);

CREATE TABLE sections (
  id uuid PRIMARY KEY,
  course_id uuid NOT NULL REFERENCES courses(id),
  term_id uuid NOT NULL REFERENCES terms(id),
  section_number text NOT NULL,
  owning_org_unit_id uuid NOT NULL,
  capacity integer NOT NULL CHECK (capacity > 0),
  workflow_state text NOT NULL CHECK (
    workflow_state IN (
      'draft','validation_pending','validation_failed','department_ready',
      'chair_review','dean_review','registrar_review','room_coordination',
      'academic_affairs_review','approved_for_publish','published',
      'change_requested','locked','denied','archived'
    )
  ),
  exception_flag boolean NOT NULL DEFAULT false,
  last_submitted_at timestamptz,
  published_at timestamptz,
  locked_at timestamptz,
  created_by uuid NOT NULL,
  updated_by uuid NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE(course_id, term_id, section_number)
);

CREATE TABLE section_meetings (
  id uuid PRIMARY KEY,
  section_id uuid NOT NULL REFERENCES sections(id),
  day_of_week smallint NOT NULL CHECK (day_of_week BETWEEN 1 AND 7),
  start_time time NOT NULL,
  end_time time NOT NULL,
  room_id uuid,
  instructor_id uuid,
  date_start date NOT NULL,
  date_end date NOT NULL
);

CREATE TABLE workflow_instances (
  id uuid PRIMARY KEY,
  entity_type text NOT NULL CHECK (entity_type IN ('section','term_calendar_event')),
  entity_id uuid NOT NULL,
  current_state text NOT NULL,
  initiated_by uuid NOT NULL,
  started_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE workflow_tasks (
  id uuid PRIMARY KEY,
  workflow_instance_id uuid NOT NULL REFERENCES workflow_instances(id),
  assigned_role_code text NOT NULL,
  assigned_scope_type text NOT NULL,
  assigned_scope_id uuid,
  assigned_user_id uuid,
  task_type text NOT NULL,
  status text NOT NULL CHECK (status IN ('open','approved','returned','denied','cancelled')),
  due_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now(),
  resolved_at timestamptz
);
```

```
CREATE TABLE workflow_transitions (
  id uuid PRIMARY KEY,
  workflow_instance_id uuid NOT NULL REFERENCES workflow_instances(id),
  from_state text NOT NULL,
  to_state text NOT NULL,
  action text NOT NULL,
  acted_by uuid NOT NULL,
  acted_as_role_code text NOT NULL,
  justification text,
  metadata jsonb NOT NULL DEFAULT '{}'::jsonb,
  created_at timestamptz NOT NULL DEFAULT now()
);
```

Conflict and validation schema

```
CREATE TABLE validation_runs (
  id uuid PRIMARY KEY,
  entity_type text NOT NULL CHECK (entity_type IN ('section','term')),
  entity_id uuid NOT NULL,
  trigger_type text NOT NULL CHECK (trigger_type IN ('save','submit','transition','prepublish','batch')
  status text NOT NULL CHECK (status IN ('passed','failed','warning_only')),
  blocking_count integer NOT NULL DEFAULT 0,
  warning_count integer NOT NULL DEFAULT 0,
  created_by uuid,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE scheduling_conflicts (
  id uuid PRIMARY KEY,
  term_id uuid NOT NULL REFERENCES terms(id),
  conflict_type text NOT NULL CHECK (
    conflict_type IN (
      'instructor_overlap','room_overlap','capacity_mismatch',
      'pattern_violation','contact_hour_mismatch','feature_mismatch',
      'cross_list_mismatch'
    )
  ),
  severity text NOT NULL CHECK (severity IN ('warning','blocking')),
  left_section_id uuid NOT NULL REFERENCES sections(id),
  right_section_id uuid REFERENCES sections(id),
  room_id uuid,
  instructor_id uuid,
  status text NOT NULL CHECK (status IN ('open','waived','resolved')),
  opened_by_run_id uuid REFERENCES validation_runs(id),
  waiver_reason text,
  waived_by uuid,
  resolved_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now()
);
```

RBAC schema

```
CREATE TABLE roles (
  id uuid PRIMARY KEY,
  code text UNIQUE NOT NULL,
  name text NOT NULL
);

CREATE TABLE permissions (
  id uuid PRIMARY KEY,
  resource text NOT NULL,
  action text NOT NULL,
  UNIQUE(resource, action)
);
```

```
CREATE TABLE role_permissions (
  role_id uuid NOT NULL REFERENCES roles(id),
  permission_id uuid NOT NULL REFERENCES permissions(id),
  PRIMARY KEY (role_id, permission_id)
);

CREATE TABLE role_bindings (
  id uuid PRIMARY KEY,
  user_id uuid NOT NULL,
  role_id uuid NOT NULL REFERENCES roles(id),
  scope_type text NOT NULL CHECK (
    scope_type IN ('institution','campus','college','department','program','room_pool')
  ),
  scope_id uuid NOT NULL,
  starts_at timestamptz,
  ends_at timestamptz,
  granted_by uuid NOT NULL,
  justification text,
  created_at timestamptz NOT NULL DEFAULT now()
);
```

Audit and FERPA tracking schema

```
CREATE TABLE audit_events (
  id uuid PRIMARY KEY,
  actor_user_id uuid NOT NULL,
  acted_as_role_code text NOT NULL,
  module text NOT NULL,
  action text NOT NULL,
  target_type text NOT NULL,
  target_id uuid NOT NULL,
  scope_type text NOT NULL,
  scope_id uuid NOT NULL,
  reason_code text,
  legitimate_interest text,
  metadata jsonb NOT NULL DEFAULT '{}'::jsonb,
  ip_address inet,
  user_agent text,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE ferpa_disclosure_requests (
  id uuid PRIMARY KEY,
  student_id uuid NOT NULL,
  requester_type text NOT NULL,
  requester_identity text NOT NULL,
  requested_record_type text NOT NULL,
  purpose text NOT NULL,
  status text NOT NULL CHECK (status IN ('submitted','under_review','approved','denied','fulfilled')),
  reviewed_by uuid,
  decision_reason text,
  created_at timestamptz NOT NULL DEFAULT now(),
  decided_at timestamptz,
  fulfilled_at timestamptz
);

CREATE TABLE ferpa_disclosure_log (
  id uuid PRIMARY KEY,
  disclosure_request_id uuid REFERENCES ferpa_disclosure_requests(id),
  student_id uuid NOT NULL,
  disclosed_by uuid NOT NULL,
  record_type text NOT NULL,
  recipient_identity text NOT NULL,
  disclosure_basis text NOT NULL,
  metadata jsonb NOT NULL DEFAULT '{}'::jsonb,
```

```
  created_at timestamptz NOT NULL DEFAULT now()
);
```

Audit logging and FERPA tracking

FERPA-focused guidance consistently emphasizes recording who accessed student records, when, which records were involved, what action occurred, and why access or disclosure was allowed. It also distinguishes general audit logs from disclosure records, which should be tracked separately. [13] [14] [6]

Required audit events

At minimum, log:

- Successful and failed sign-ins.
- Record views involving FERPA-sensitive student data.
- Section create/edit/delete actions.
- Workflow approvals, returns, denials, and reroutes.
- Publish, lock, unlock, and override actions.
- Data exports and download events.
- Permission changes and role grants.
- Disclosure-request decisions.
- Vendor syncs or outbound transfers where student data is included.

Minimum audit fields

Field	Why it matters
Actor identity	Who performed the action
Role used	Which permission path allowed it
Timestamp	When it occurred
Module and action	What the user did
Target record(s)	Which records were touched
Scope	Which department/campus/college context applied
Legitimate interest / reason	Why access was justified
IP / client details	For investigations and anomaly review
Result	Allowed, denied, failed, returned, exported

FERPA tracking rules

- Log record access for FERPA-relevant data, not just login events. Sources warn that logging only authentication is not enough. [6]
- Separate **access logs** from **disclosure logs** because they serve different compliance purposes. [13]
- Require a business-need reason for exports and sensitive overrides.
- Maintain append-only or tamper-evident audit storage for high-risk events. Write-once or append-only logging is recommended in FERPA-focused tooling guidance. [7]
- Schedule periodic access reviews to confirm each role still has a legitimate educational purpose. [15] [6]
- Track permission changes with approver, date, and justification, since access ownership and revocation are part of effective FERPA governance. [15]

Endpoint-to-audit examples

Endpoint	Audit event	FERPA relevance
`POST /api/sections/{id}/submit`	`workflow.submit`	Low unless student-linked fields included
`POST /api/workflow/tasks/{id}/approve`	`workflow.approve`	Moderate, official decision trail
`POST /api/terms/{id}/publish`	`schedule.publish`	High, official record action
`POST /api/exports/sections`	`records.export`	High, requires reason and scope
`GET /api/audit-events?...`	`audit.view`	High, compliance-only visibility
`POST /api/ferpa/disclosure-requests/{id}/decide`	`ferpa.disclosure_decision`	High, compliance evidence

Recommended implementation pattern

Three design choices make this robust:

- Keep workflow transitions append-only even if the current state is denormalized on `sections`.
- Enforce RBAC through role templates plus scoped bindings, not hardcoded page checks.
- Treat audit and FERPA evidence as first-class domains, not debug logs.

That design gives Semester a registrar-grade workflow engine with Academic Affairs routing, formal publication control, and defensible access evidence for sensitive record handling. [2] [1] [6]

⁂

1. https://it.osu.edu/student-information-system-sis/sis-room-class-scheduling/1-course-class-workflow
2. https://www.aacrao.org/news/electronic-workflows-to-radically-support-graduate-education/
3. https://public.powerdms.com/CSUChico/documents/2348080
4. https://www.illumio.com/cybersecurity-101/rbac
5. https://www.uncsa.edu/information-technology/standards/access-control.aspx
6. https://openeducat.org/articles/ferpa-compliance-education-software/
7. https://secureprivacy.ai/blog/ferpa-compliance-software
8. https://registrar.vanderbilt.edu/workflow.php
9. https://registrar.uiowa.edu/faculty-and-staff/classroom-scheduling/course-scheduling-policies-and-procedures
10. https://coursedog.com/articles/best-practices-for-strategic-scheduling
11. https://www.unomaha.edu/registrar/faculty-and-staff/clss/course-schedule-building-guidelines.php
12. https://www.getinsight.space/resources/ferpa-compliance-software-guide
13. https://sftptogo.com/blog/secure-cloud-sftp-for-ferpa/
14. https://edves.us/compliance/ferpa-and-student-data-privacy/
15. https://nhimg.org/faq/how-should-educational-institutions-reduce-the-risk-of-ferpa-violations-when-sta/
16. https://registrar.vanderbilt.edu/faculty-staff/academic-scheduling.php